

软件学院 2024-2025 学年度第一学期

《软件设计综合实践》实验报告

班级：	软件工程 230X 班
学号：	U202317XXX
姓名：	XXX

目录

1. 实验内容和要求	3
2.斐波拉契数代码	3
3.词法分析程序	4
3.1 主要设计和实现思路	4
3.2 词法分析程序代码	7
3.3 实验演示	18
4.语法分析程序	22
4.1 主要设计和实现思路	22
4.2 递归下降语法分析程序代码	25
4.3 实验演示	47
5.语义分析程序	51
5.1 主要设计和实现思路	51
5.2 类型检查语义分析程序代码	53
5.3 实验演示	76
6.实验总结	78

1. 实验内容和要求

1. 用 C--语言写一个函数计算第 n（从标准输入获得）个斐波那契数，并将计算结果输出到屏幕；
2. 用 C 语言设计和实现 C--语言的词法分析器，并对输入的 C--代码输出词法分析扫描结果；
3. 用 C 语言设计和实现 C--语言的递归下降语法分析器，并对输入的 C--代码输出语法树。
4. 用 C 语言设计和实现对 C--语言进行符号表构造和类型检查的语义分析程序。

2. 斐波拉契数代码

```
int fibonacci(int n){
    int sum;
    int i;
    int first;
    int second;

    sum = 0;
    i = 2;
    first = 1;
    second = 1;

    if (n == 1){
        return first;
    }

    else if(n == 2){
        return second;
    }

    else{
        while(i < n){
            sum = first + second;
            first = second;
            second = sum;
            i = i + 1;
        }
    }
}
```

```

        return sum;
    }

void main(void){
    int n;
    n = input();
    output(fibonacci(n));
}

```

3.词法分析程序

3.1 主要设计和实现思路

1.由《编译原理及实践》附录 A，C--的词法如下：

1. 下面是语言的关键字：

```
else if int return void while
```

所有的关键字都是保留字，并且必须是小写。

2. 下面是专用符号：

```
+ - * / < <= > >= == != = ; , ( ) [ ] { } /* */
```

3. 其他标记是**ID**和**NUM**，通过下列正则表达式定义：

```
ID = letter letter*
NUM = digit digit*
letter = a|..|z|A|..|Z
digit = 0|..|9
```

小写和大写字母是有区别的。

4. 空格由空白、换行符和制表符组成。空格通常被忽略，除了它必须分开**ID**、**NUM**关键字。

5. 注释用通常的C语言符号**/*...*/**围起来。注释可以放在任何空白出现的位置(即注释不能放在标记内)上，且可以超过一行。注释不能嵌套。

注：由于无数组，似乎不需要[]，故我并未将[]列入 token 考虑；

2.首先定义 C--中所有的记号(token)标记为枚举(enum)类型的逻辑项：

```
typedef enum{
    /* book-keeping tokens */
    ENDOFFILE, ERROR,
    /* reserved words */
    IF, ELSE, WHILE, RETURN, VOID, INT,
    /* multicharacter tokens */
    ID, NUM,
    /* special symbols */

```

```
    ASSIGN, EQ, NE, LT, GT, LTE, GTE, PLUS, MINUS, TIMES, OVER, LPAREN, RPAREN, LBRACE, RBRACE, SEMI,  
    COMMA  
}TokenType;
```

3.我们使用状态机识别 token:

```
typedef enum{  
    START, INNUM, INID, INLT, INGT, INEQ, INNE, INDIV, INCOMMENT, ENDCOMMENT, DONE  
}StateType;
```

4.储存 C—的关键字（也是保留字）：

```
static struct{  
    char* str;  
    TokenType tok;  
} reservedWords[MAXRESERVED]  
= { {(char*)"if", IF}, {(char*)"else", ELSE},  
    {(char*)"while", WHILE}, {(char*)"int", INT},  
    {(char*)"void", VOID}, {(char*)"return", RETURN} };
```

5.使用 getToken(void)根据状态码获取当前 token，具体算法描述如下：

(1)初始化：

tokenStringIndex: 用于在 tokenString 中存储当前识别的标记。

currentToken: 用于存储当前识别的标记类型。

state: 当前的状态，初始为 START。

save: 标志位，指示是否将当前字符保存到 tokenString 中。

(2)循环处理：直到状态变为 DONE，循环执行以下步骤：

调用 getNextChar()获取下一个非空白字符 c。

根据当前状态 state，执行不同的操作。

(3)状态机：

START: 根据字符 c 的类型，转换到相应的状态或直接设置 currentToken。

数字：转到 INNUM 状态。

字母：转到 INID 状态。

=：转到 INEQ 状态。

<：转到 INLT 状态。

>：转到 INGT 状态。

!：转到 INNE 状态。

/：转到 INDIV 状态。

空白字符：不保存，继续循环。

其他特殊字符：设置 currentToken 并结束状态机。

INEQ：检查是否为==，如果是，设置 currentToken 为 EQ；否则，回退一个字符，设置为 ASSIGN。

INLT：检查是否为<=，如果是，设置 currentToken 为 LTE；否则，回退一个字符，设置为 LT。

INGT：检查是否为>=，如果是，设置 currentToken 为 GTE；否则，回退一个字符，设置为 GT。

INNE：检查是否为!=，如果是，设置 currentToken 为 NE；否则，回退一个字符，设置为 ERROR。

INDIV：检查是否为*，如果是，转到 INCOMMENT 状态；否则，回退一个字符，设置为 TIMES。

INCOMMENT：忽略字符，直到遇到*，然后转到 ENDCOMMENT 状态。

ENDCOMMENT：检查是否为/，如果是，回到 START 状态；否则，如果遇到*，保持在 ENDCOMMENT 状态；否则，回到 INCOMMENT 状态。

INNUM：读取数字字符，直到遇到非数字字符，然后设置 currentToken 为 NUM。

INID：读取字母和数字字符，直到遇到非字母数字字符，然后设置 currentToken

为 ID。

(4)保存标记：如果 save 为真且 tokenStringIndex 小于 MAXTOKENLEN，则将字符 c 添加到 tokenString 中。

(5)结束处理：当状态变为 DONE 时，将 tokenString 的当前索引位置设置为字符串结束符\0。如果 currentToken 是 ID，则调用 reservedLookup 检查是否为保留字。

(6)跟踪输出：如果 TraceScan 为真，调用 printToken()输出 currentToken 和对应的 tokenString。

(7)返回标记：返回 currentToken。

（在 main.c 文件中将 NO_PARSE 和 TraceScan 设置为 TRUE 即可演示）

3.2 词法分析程序代码

注：main.c 放在本文档最后

globals.h(仅为其中部分，后续还有完善):

```
/* **** */
/* File: globals.h */
/* Global types and vars for C++ compiler */
/* must come before other include files */
/* START: 2024.11.25 */
/* FINISH: 2024.12.16 */
/* **** */

#ifndef _GLOBALS_H_
#define _GLOBALS_H_

#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <assert.h>

#ifndef FALSE
#define FALSE 0
#endif
```

```

#ifndef TRUE
#define TRUE 1
#endif

/* A useful debugging macro */
#ifdef DEBUG
#define DEBUG_ONLY(X) X
#else
#define DEBUG_ONLY(X)
#endif

/* MAXRESERVED = the number of reserved words */
#define MAXRESERVED 6

typedef enum
{
    /* book-keeping tokens */
    ENDOFFILE, ERROR,
    /* reserved words */
    IF, ELSE, WHILE, RETURN, VOID, INT,
    /* multicharacter tokens */
    ID, NUM,
    /* special symbols */
    ASSIGN, EQ, NE, LT, GT, LTE, GTE, PLUS, MINUS, TIMES, OVER, LPAREN, RPAREN, LBRACE, RBRACE, SEMI,
    COMMA
} TokenType;

extern FILE* source; /* source code text file */
extern FILE* listing; /* listing output text file */
extern FILE* code; /* code text file for TM simulator */

extern int lineno; /* source line number for listing */

/*****
*****      Flags for tracing      *****/
/*****
*****
*****/

/* EchoSource = TRUE causes the source program to
 * be echoed to the listing file with line numbers
 * during parsing
 */
extern int EchoSource;

/* TraceScan = TRUE causes token information to be
 * printed to the listing file as each token is

```



```

    * recognized by the scanner
    */
extern int TraceScan;

/* TraceParse = TRUE causes the syntax tree to be
 * printed to the listing file in linearized form
 * (using indents for children)
 */
extern int TraceParse;

/* TraceAnalyze = TRUE causes symbol table inserts
 * and lookups to be reported to the listing file
 */
extern int TraceAnalyze;

/* Error = TRUE prevents further passes if an error occurs */
extern int Error;

#endif

```

util.h(仅为该文件部分，后续还有完善):

```

/*****
 * File: util.h
 * Utility functions for the C-- compiler
 * 2024.11.25
 *****/

#ifndef _UTIL_H_
#define _UTIL_H_

/* Procedure printToken prints a token
 * and its lexeme to the listing file
 */
void printToken(TokenType, const char*);

#endif

```

util.c(仅为该文件部分，后续还有完善):

```

/*****
 * File: util.c
 * Utility function implementation
 */

```

```

/* for the C-- compiler                                     */
/* 2024.11.25                                              */
/*****/

#include "globals.h"
#include "util.h"

/* Procedure printToken prints a token
 * and its lexeme to the listing file
 */
void printToken(TokenType token, const char* tokenString)
{
    switch (token)
    {
        case IF:
        case ELSE:
        case WHILE:
        case RETURN:
        case VOID:
        case INT:
            fprintf(listing,
                    "reserved word: %s\n", tokenString);
            break;
        case ASSIGN: fprintf(listing, "=\n"); break;
        case LT: fprintf(listing, "<\n"); break;
        case GT: fprintf(listing, ">\n"); break;
        case EQ: fprintf(listing, "=\n"); break;
        case NE: fprintf(listing, "!=\n"); break;
        case LPAREN: fprintf(listing, "(\n"); break;
        case RPAREN: fprintf(listing, ")\n"); break;
        case LBRACE: fprintf(listing, "{\n"); break;
        case RBRACE: fprintf(listing, "}\n"); break;
        case LTE: fprintf(listing, "<=\n"); break;
        case GTE: fprintf(listing, ">=\n"); break;
        case SEMI: fprintf(listing, ";\n"); break;
        case COMMA: fprintf(listing, ",\n"); break;
        case PLUS: fprintf(listing, "+\n"); break;
        case MINUS: fprintf(listing, "-\n"); break;
        case TIMES: fprintf(listing, "*\n"); break;
        case OVER: fprintf(listing, "/\n"); break;
        case ENDOFFILE: fprintf(listing, "EOF\n"); break;
        case NUM:
            fprintf(listing,
                    "NUM, val= %s\n", tokenString);
    }
}

```

```

        break;
    case ID:
        fprintf(listing,
            "ID, name= %s\n", tokenString);
        break;
    case ERROR:
        fprintf(listing,
            "ERROR: %s\n", tokenString);
        break;
    default: /* should never happen */
        fprintf(listing, "Unknown token: %d\n", token);
    }
}

```

scan.h(对应扫描程序):

```

/*****
/* File: scan.h                                     */
/* The scanner interface for the C-- compiler      */
/* START: 2024.11.25                               */
/* FINISH: 2024.12.2                               */
*****/

#ifndef _SCAN_H_
#define _SCAN_H_

/* MAXTOKENLEN is the maximum size of a token */
#define MAXTOKENLEN 40

/* tokenString array stores the lexeme of each token */
extern char tokenString[MAXTOKENLEN + 1];

/* function getToken returns the
 * next token in source file
 */
TokenType getToken(void);

#endif

```

scan.c(对应扫描程序):

```

/*****
/* File: scan.c                                     */

```

```

/* The scanner implementation for the C-- compiler */
/* START: 2024.11.25 */
/* FINISH: 2024.12.9 */
/*****/

#include "globals.h"
#include "util.h"
#include "scan.h"

/* states in scanner DFA */
typedef enum
{
    START, INNUM, INID, INLT, INGT, INEQ, INNE, INDIV, INCOMMENT, ENDCOMMENT, DONE
}
StateType;

/* lexeme of identifier or reserved word */
char tokenString[MAXTOKENLEN + 1];

/* BUFLen = length of the input buffer for
   source code lines */
#define BUFLen 256

static char lineBuf[BUFLen]; /* holds the current line */
static int linepos = 0; /* current position in LineBuf */
static int bufsize = 0; /* current size of buffer string */
static int EOF_flag = FALSE; /* corrects ungetNextChar behavior on EOF */

/* getNextChar fetches the next non-blank character
   from lineBuf, reading in a new line if lineBuf is
   exhausted */
static int getNextChar(void)
{
    if (!(linepos < bufsize))
    {
        lineno++;
        if (fgets(lineBuf, BUFLen - 1, source))
        {
            if (EchoSource) fprintf(listing, "%4d: %s", lineno, lineBuf);
            bufsize = strlen(lineBuf);
            linepos = 0;
            return lineBuf[linepos++];
        }
    }
    else

```

```

        {
            EOF_flag = TRUE;
            return EOF;
        }
    }
    else return lineBuf[linepos++];
}

/* ungetNextChar backtracks one character
   in lineBuf */
static void ungetNextChar(void)
{
    if (!EOF_flag) linepos--;
}

/* lookup table of reserved words */
static struct
{
    char* str;
    TokenType tok;
} reservedWords[MAXRESERVED]
= { {(char*)"if", IF}, {(char*)"else", ELSE},
    {(char*)"while", WHILE}, {(char*)"int", INT},
    {(char*)"void", VOID}, {(char*)"return", RETURN} };

/* lookup an identifier to see if it is a reserved word */
/* uses linear search */
static TokenType reservedLookup(char* s)
{
    int i;
    for (i = 0; i < MAXRESERVED; i++)
        if (!strcmp(s, reservedWords[i].str))
            return reservedWords[i].tok;
    return ID;
}

/*****
/* the primary function of the scanner */
*****/

/* function getToken returns the
   * next token in source file
   */
TokenType getToken(void)
{ /* index for storing into tokenString */

```

```

int tokenStringIndex = 0;
/* holds current token to be returned */
TokenType currentToken;
/* current state - always begins at START */
StateType state = START;
/* flag to indicate save to tokenString */
int save;
while (state != DONE)
{
    int c = getNextChar();
    save = TRUE;
    switch (state)
    {
    case START:
        if (isdigit(c))
            state = INNUM;
        else if (isalpha(c))
            state = INID;
        else if (c == '=')
            state = INEQ;
        else if (c == '<')
            state = INLT;
        else if (c == '>')
            state = INGT;
        else if (c == '!')
            state = INNE;
        else if (c == '/')
            state = INDIV;
        else if ((c == ' ') || (c == '\t') || (c == '\n'))
            save = FALSE;
        else
        {
            state = DONE;
            switch (c)
            {
            case EOF:
                save = FALSE;
                currentToken = ENDOFFILE;
                break;
            case '+':
                currentToken = PLUS;
                break;
            case '-':
                currentToken = MINUS;

```

```

        break;
    case '*':
        currentToken = TIMES;
        break;
    case '(':
        currentToken = LPAREN;
        break;
    case ')':
        currentToken = RPAREN;
        break;
    case ';':
        currentToken = SEMI;
        break;
    case ',':
        currentToken = COMMA;
        break;
    case '{':
        currentToken = LBRACE;
        break;
    case '}':
        currentToken = RBACE;
        break;
    default:
        currentToken = ERROR;
        break;
    }
}
break;

case INEQ:
    state = DONE;
    if (c == '=')
        currentToken = EQ;
    else
    { /* backup in the input */
        ungetNextChar();
        save = FALSE;
        currentToken = ASSIGN;
    }
    break;

case INLT:
    state = DONE;
    if (c == '=')
        currentToken = LTE;

```

```

else
{
    ungetNextChar();
    save = FALSE;
    currentToken = LT;
}
break;
case INGT:
    state = DONE;
    if (c == '=')
        currentToken = GTE;
    else
    {
        ungetNextChar();
        save = FALSE;
        currentToken = GT;
    }
    break;
case INNE:
    state = DONE;
    if (c == '=')
        currentToken = NE;
    else
    {
        ungetNextChar();
        save = FALSE;
        currentToken = ERROR;
    }
    break;
case INDIV:
    if (c == '*')
    {
        save = FALSE;
        state = INCOMMENT;
        tokenStringIndex -= 1;
    }
    else
    {
        ungetNextChar();
        save = FALSE;
        state = DONE;
        currentToken = TIMES;
    }
    break;

```



```

case INCOMMENT:
    save = FALSE;
    if (c == '*')
        state = ENDCOMMENT;
    break;

case ENDCOMMENT:
    save = FALSE;
    if (c == '/')
        state = START;
    else if (c == '*')
        state = ENDCOMMENT;
    else
        state = INCOMMENT;
    break;

case INNUM:
    if (!isdigit(c))
    { /* backup in the input */
        ungetNextChar();
        save = FALSE;
        state = DONE;
        currentToken = NUM;
    }
    break;

case INID:
    if (!isalpha(c))
    { /* backup in the input */
        ungetNextChar();
        save = FALSE;
        state = DONE;
        currentToken = ID;
    }
    break;

case DONE:
default: /* should never happen */
    fprintf(listing, "Scanner Bug: state= %d\n", state);
    state = DONE;
    currentToken = ERROR;
    break;
}

// append new id to token string
if ((save) && (tokenStringIndex <= MAXTOKENLEN))

```

```

        tokenString[tokenStringIndex++] = (char)c;
    if (state == DONE)
    {
        tokenString[tokenStringIndex] = '\0';
        if (currentToken == ID)
            currentToken = reservedLookup(tokenString);
    }
}
if (TraceScan)
{
    fprintf(listing, "\t%d: ", lineno);
    printToken(currentToken, tokenString);
}
return currentToken;
} /* end getToken */

```

3.3 实验演示

求三个整数中的最大值：

```
Microsoft Visual Studio 调 × + v

C-- COMPILATION: Maximum.txt
2: reserved word: int
2: ID, name= max
2: (
2: reserved word: int
2: ID, name= x
2: ,
2: reserved word: int
2: ID, name= y
2: ,
2: reserved word: int
2: ID, name= z
2: )
3: {
4: reserved word: int
4: ID, name= biggest
4: ;
5: ID, name= biggest
5: =
5: ID, name= x
5: ;
6: reserved word: if
6: (
6: ID, name= y
6: >
6: ID, name= biggest
6: )
7: ID, name= biggest
7: =
7: ID, name= y
7: ;
8: reserved word: if
8: (
8: ID, name= z
8: >
8: ID, name= biggest
8: )
8: ID, name= biggest
8: =
8: ID, name= z
8: ;
10: reserved word: return
10: ID, name= biggest
10: ;
11: }
13: reserved word: void
13: ID, name= main
13: (
13: reserved word: void
13: )
14: {
```

```
15: reserved word: int
15: ID, name= x
15: ;
16: reserved word: int
16: ID, name= y
16: ;
17: reserved word: int
17: ID, name= z
17: ;
18: reserved word: int
18: ID, name= biggest
18: ;
20: ID, name= x
20: =
20: ID, name= input
20: (
20: )
20: ;
21: ID, name= y
21: =
21: ID, name= input
21: (
21: )
21: ;
22: ID, name= z
22: =
22: ID, name= input
22: (
22: )
22: ;
24: ID, name= biggest
24: =
24: ID, name= max
24: (
24: ID, name= x
24: ,
24: ID, name= y
24: ,
24: ID, name= z
24: )
24: ;
25: ID, name= output
25: (
25: ID, name= biggest
25: )
25: ;
26: }
27: EOF

D:\visual studio\Compiler\x64\Debug\Com
```

求从 1 到 N 的累加:

```
Microsoft Visual Studio 调 × + v

C-- COMPILATION: Sum.txt
1: reserved word: int
1: ID, name= sum
1: (
1: reserved word: int
1: ID, name= n
1: )
2: {
3: reserved word: int
3: ID, name= result
3: ;
4: reserved word: int
4: ID, name= i
4: ;
6: ID, name= i
6: =
6: NUM, val= 1
6: ;
7: ID, name= result
7: =
7: NUM, val= 0
7: ;
9: reserved word: while
9: (
9: ID, name= i
9: <= 9: ID, name= n
9: )
10: {
11: ID, name= result
11: =
11: ID, name= result
11: +
11: ID, name= i
11: ;
12: ID, name= i
12: =
12: ID, name= i
12: +
12: NUM, val= 1
12: ;
13: }
15: reserved word: return
15: ID, name= result
15: ;
16: }
18: reserved word: void
18: ID, name= main
18: (
18: reserved word: void
18: )
19: {
20: reserved word: int
20: ID, name= n
20: ;
21: reserved word: int
21: ID, name= s
21: ;
23: ID, name= n
23: =
23: ID, name= input
23: (
23: )
23: ;
24: ID, name= s
24: =
24: ID, name= sum
24: (
24: ID, name= n
24: )
24: ;
25: ID, name= output
25: (
25: ID, name= s
25: )
25: ;
26: }
27: EOF

D:\visual studio\Compiler\x64\Debug\
要在调试停止时自动关闭控制台, 请启用
按任意键关闭此窗口. . .|
```

斐波那契数列代码:

```
Microsoft Visual Studio 调 × + ∨

C-- COMPILATION: Fibonacci.txt
1: reserved word: int
1: ID, name= fibonacci
1: (
1: reserved word: int
1: ID, name= n
1: )
1: {
2: reserved word: int
2: ID, name= sum
2: ;
3: reserved word: int
3: ID, name= i
3: ;
4: reserved word: int
4: ID, name= first
4: ;
5: reserved word: int
5: ID, name= second
5: ;
7: ID, name= sum
7: =
7: NUM, val= 0
7: ;
8: ID, name= i
8: =
8: NUM, val= 2
8: ;
9: ID, name= first
9: =
9: NUM, val= 1
9: ;
10: ID, name= second
10: =
10: NUM, val= 1
10: ;
12: reserved word: if
12: (
12: ID, name= n
12: ==
12: NUM, val= 1
12: )
12: {
13: reserved word: return
13: ID, name= first
13: ;
14: }
16: reserved word: else
16: reserved word: if
16: (
16: ID, name= n
```

```
Microsoft Visual Studio 调 × + ∨

16: ID, name= n
16: ==
16: NUM, val= 2
16: )
16: {
17: reserved word: return
17: ID, name= second
17: ;
18: }
20: reserved word: else
20: {
21: reserved word: while
21: (
21: ID, name= i
21: <
21: ID, name= n
21: )
21: {
22: ID, name= sum
22: =
22: ID, name= first
22: +
22: ID, name= second
22: ;
23: ID, name= first
23: =
23: ID, name= second
23: ;
24: ID, name= second
24: =
24: ID, name= sum
24: ;
25: ID, name= i
25: =
25: ID, name= i
25: +
25: NUM, val= 1
25: ;
26: }
27: }
28: reserved word: return
28: ID, name= sum
28: ;
29: }
31: reserved word: void
31: ID, name= main
31: (
31: reserved word: void
31: )
31: {
32: reserved word: int
32: ID, name= n
```

```

32: reserved word: int
32: ID, name= n
32: ;
33: ID, name= n
33: =
33: ID, name= input
33: (
33: )
33: ;
34: ID, name= output
34: (
34: ID, name= fibonacci
34: (
34: ID, name= n
34: )
34: )
34: ;
35: }
36: EOF

```

D:\visual_studio\Compiler\x64\Deb
要在调试停止时自动关闭控制台，请启
按任意键关闭此窗口 . . .|

4. 语法分析程序

4.1 主要设计和实现思路

1.C—共有三种基本结构：语句、表达式和声明。故可将语法树节点定为这三类。
其中语句共有：if,while,return,call,compound；表达式共有：算符表达式、常量表达式、标识符表达式和赋值表达式；声明共有：变量声明和函数声明。

```

typedef enum { StmtK, ExpK, DecK } NodeKind;
typedef enum { IfK, WhileK, ReturnK, CallK, CompoundK } StmtKind;
typedef enum { OpK, IdK, ConstK, AssignK } ExpKind;
typedef enum { ScalarDecK, FuncDecK } DecKind;
typedef enum { Void, Integer, Function } ExpType;

```

2.语法树节点可按上面分类，由此定义语法树(Syntax tree)，并进行节点类型的初始化。它包含以下字段：

child[MAXCHILDREN]：指向子节点的指针数组，最大子节点数为 3。

sibling: 指向兄弟节点的指针。

lineno: 节点在源代码中的行号。

nodekind: 节点类型, 可以是语句 (**StmtK**)、表达式 (**ExpK**) 或声明 (**DecK**)。

kind: 一个联合体, 根据 **nodekind** 的值, 可以是 **StmtKind**、**ExpKind** 或 **DecKind**。

op: 操作符类型, 用于表达式节点。

val: 整数值, 用于常量表达式节点。

name: 标识符名称, 用于标识符和函数声明节点。

functionReturnType、**variableDataType**、**expressionType:** 用于类型检查的字段, 分别表示函数返回类型、变量数据类型和表达式类型。

isGlobal、**isParameter:** 标志位, 表示变量是否为全局变量或函数参数。

declaration: 指向声明节点的指针, 用于变量或函数的声明。

3. 参考书中附录 A 给出 C-- 的 BNF 描述, 可为文法中的每个非终结符编写一个函数, 由这些函数递归地构建树。解释详见《编译原理及实践》附录 A:

1. **program** → **declaration-list**
2. **declaration-list** → **declaration-list declaration** | **declaration**
3. **declaration** → **var-declaration** | **fun-declaration**
4. **var-declaration** → **type-specifier ID**;
5. **type-specifier** → **int** | **void**
6. **fun-declaration** → **type-specifierID(params)** | **compound-stmt**
7. **params** → **params-list** | **void**
8. **param-list** → **param-list , param** | **param**
9. **param** → **type-specifierID**
10. **compound-stmt** → {**local-declarationsstatement-list**}
11. **local-declarations** → **local-declarationsvar-declaration** | **empty**
12. **statement-list** → **statement-liststatement** | **empty**
13. **statement** → **expression-stmt**
 | **compound-stmt**
 | **selection-stmt**
 | **iteration-stmt**
 | **return-stmt**

- 14. expression-stmt $\rightarrow$ expression; | ;
- 15. selection-stmt $\rightarrow$ if(expression)statement
| if (expression)statement else statement
- 16. iteration-stmt $\rightarrow$ while(expression)statement
- 17. return-stmt $\rightarrow$ return; | return expression;
- 18. expression $\rightarrow$ var = expression | simple-expression
- 19. var $\rightarrow$ ID
- 20. simple-expression $\rightarrow$ additive-expression relop additive-expression
| additive-expression
- 21. relop $\rightarrow$ <= | < | > | >= | == | !=
- 22. additive-expression $\rightarrow$ additive-expression addop term | term
- 23. addop $\rightarrow$ + | -
- 24. term $\rightarrow$ term mulop factor | factor
- 25. mulop $\rightarrow$ * | /
- 26. factor $\rightarrow$ (expression) | var | call | NUM
- 27. call $\rightarrow$ ID(args)
- 28. args $\rightarrow$ arg-list | empty
- 29. arg-list $\rightarrow$ arg-list, expression | expression

4.根据 BNF 范式，如果后一语句不为空，则将前一语句作为后一语句的树根，将他们全部连接起来，然后依次向下进行递归调用，根据相应的函数进行分析，构建语法树。主要函数包括：

parse(): 开始解析过程，返回根节点。

declaration_list(): 解析一系列声明。

declaration(): 解析单个声明，可以是变量声明或函数声明。

statement_list(): 解析一系列语句。

statement(): 解析单个语句，可以是复合语句、if 语句、while 语句、return 语句或表达式语句。

expression(): 解析表达式，可以是赋值表达式、简单表达式、加法表达式、项或因子。

5.分析结束后，调用 util.c 中的 printTree()函数进行语法树的输出；

6.错误处理：在解析函数中，当遇到意外的标记时，调用 syntaxError()函数输出

错误信息。使用 `match()` 函数来确保当前标记与预期的标记相匹配, 如果不匹配, 则报告错误;

(在 `main.c` 中将 `NO_ANALYZE` 和 `TraceParse` 设置为 `TRUE` 即可演示实验)

4.2 递归下降语法分析程序代码

`globals.h`:

```
/******  
***** Syntax tree for parsing *****  
*****  
  
typedef enum { StmtK, ExpK, DecK } NodeKind;  
/* statement kind, expression kind, declaration kind */  
typedef enum { IfK, WhileK, ReturnK, CallK, CompoundK } StmtKind;  
/* if kind, while kind, return kind, call kind, compound kind */  
typedef enum { OpK, IdK, ConstK, AssignK } ExpKind;  
/* operator kind, identifier kind, const(num) kind, assign kind */  
typedef enum { ScalarDecK, FuncDecK } DecKind;  
/* scalar declaration kind, function declaration kind */  
typedef enum { Void, Integer, Function } ExpType;  
  
#define MAXCHILDREN 3  
  
typedef struct treeNode  
{  
    struct treeNode* child[MAXCHILDREN];  
    struct treeNode* sibling;  
    int lineno;  
    NodeKind nodekind;  
    union { StmtKind stmt; ExpKind exp; DecKind dec; } kind;  
  
    TokenType op;  
    int val;  
    char* name;  
  
    /*for type checking*/  
    ExpType functionReturnType;  
    ExpType variableDataType;  
    ExpType expressionType;
```

```

/* If isGlobal is TRUE, then the variable is a global */
int isGlobal;

int isParameter;

struct treeNode* declaration;
}TreeNode;

```

util.h:

```

/* Function newStmtNode creates a new statement
 * node for syntax tree construction
 */
TreeNode* newStmtNode(StmtKind);

/* Function newExpNode creates a new expression
 * node for syntax tree construction
 */
TreeNode* newExpNode(ExpKind);

/* Function newExpNode creates a new declaration
 * node for syntax tree construction
 */
TreeNode* newDecNode(DecKind kind);

/* Function copyString allocates and makes a new
 * copy of an existing string
 */
char* copyString(char*);

char* typeName(ExpType type);
/* procedure printTree prints a syntax tree to the
 * listing file using indentation to indicate subtrees
 */
void printTree(TreeNode*);

```

util.c:

```

/* Function newStmtNode creates a new statement
 * node for syntax tree construction
 */
TreeNode* newStmtNode(StmtKind kind) {

```

```

TreeNode* node = (TreeNode*)malloc(sizeof(TreeNode));
if (node != NULL) {
    node->nodekind = StmtK;
    node->kind.stmt = kind;
    for (int i = 0; i < MAXCHILDREN; ++i) {
        node->child[i] = NULL;
    }
    node->sibling = NULL;
    node->lineno = lineno;
    node->isParameter = FALSE;
    node->isGlobal = FALSE;
    node->declaration = NULL;
}
return node;
}

/* Function newExpNode creates a new expression
 * node for syntax tree construction
 */
TreeNode* newExpNode(ExpKind kind) {
    TreeNode* node = (TreeNode*)malloc(sizeof(TreeNode));
    if (node != NULL) {
        node->nodekind = ExpK;
        node->kind.exp = kind;
        node->op = ERROR;
        node->val = 0;
        node->name = NULL;
        node->expressionType = Void;
        for (int i = 0; i < MAXCHILDREN; ++i) {
            node->child[i] = NULL;
        }
        node->sibling = NULL;
        node->lineno = lineno;
        node->isParameter = FALSE;
        node->isGlobal = FALSE;
        node->declaration = NULL;
    }
    return node;
}

/* Function newDecNode creates a new declaration
 * node for syntax tree construction
 */
TreeNode* newDecNode(DecKind kind) {
    TreeNode* node = (TreeNode*)malloc(sizeof(TreeNode));

```

```

    if (node != NULL) {
        node->nodekind = Deck;
        node->kind.dec = kind;
        node->variableDataType = Void;
        node->functionReturnType = Void;
        node->name = NULL;
        for (int i = 0; i < MAXCHILDREN; ++i) {
            node->child[i] = NULL;
        }
        node->sibling = NULL;
        node->lineno = lineno;
        node->isParameter = FALSE;
        node->isGlobal = FALSE;
        node->declaration = node;
    }
    return node;
}

/* Function copyString allocates and makes a new
 * copy of an existing string
 */
char* copyString(char* s)
{
    int n;
    char* t;
    if (s == NULL) return NULL;
    n = strlen(s) + 1;
    t = malloc(n);
    if (t == NULL)
        fprintf(listing, "Out of memory error at line %d\n", lineno);
    else strcpy(t, s);
    return t;
}

/* Variable indentno is used by printTree to
 * store current number of spaces to indent
 */
static int indentno = 0;

/* macros to increase/decrease indentation */
#define INDENT indentno+=2
#define UNINDENT indentno-=2

/* printSpaces indents by printing spaces */

```

```

static void printSpaces(void)
{
    int i;
    for (i = 0; i < indentno; i++)
        fprintf(listing, " ");
}

```

/*type check*/

```

char* typeName(ExpType type)
{
    static char i[] = "integer";
    static char v[] = "void";
    static char invalid[] = "invalid type";

    switch (type)
    {
        case Integer: return i; break;
        case Void:    return v; break;
        default:      return invalid;
    }
}

```

/* procedure printTree prints a syntax tree to the
* listing file using indentation to indicate subtrees
*/

```

void printTree(TreeNode* tree)
{
    int i;

    INDENT;

    while (tree != NULL)
    {
        printSpaces();

        if (tree->nodekind == Deck)
        {
            switch (tree->kind.dec)
            {
                case ScalarDeck:
                    fprintf(listing, "Scalar declaration: \"%s\" of type \"%s\"\\n"
                        , tree->name, typeName(tree->variableDataType));
                    break;
                case FuncDeck:

```

```

        fprintf(listing, "Function declaration:  \"%s()\\""
               " of return type \"%s\\""
               tree->name, typeName(tree->functionReturnType));
        break;
    default:
        fprintf(listing, "unknown declaration type\n");
        break;
    }
}
else if (tree->nodekind == ExpK)
{
    switch (tree->kind.exp)
    {
    case OpK:
        fprintf(listing, "Operator: ");
        printToken(tree->op, "");
        break;
    case IdK:
        fprintf(listing, "Identifier:  \"%s\"", tree->name);
        if (tree->val != 0) /* array indexing */
            fprintf(listing, "%d", tree->val);
        fprintf(listing, "\n");
        break;
    case ConstK:
        fprintf(listing, "Literal constant:  \"%d\\"", tree->val);
        break;
    case AssignK:
        fprintf(listing, "Assignment\n");
        break;
    default:
        fprintf(listing, "unknown expression type\n");
        break;
    }
}
else if (tree->nodekind == StmtK)
{
    switch (tree->kind.stmt)
    {
    case CompoundK:
        fprintf(listing, "Compound statement\n");
        break;
    case IfK:
        fprintf(listing, "IF statement\n");
        break;

```

```

        case WhileK:
            fprintf(listing, "WHILE statement\n");
            break;
        case ReturnK:
            fprintf(listing, "RETURN statement\n");
            break;
        case CallK:
            fprintf(listing, "Call to function \"%s()\"\n",
                    tree->name);
            break;
        default:
            fprintf(listing, "unknown statement type\n");
            break;
    }
}
else
    fprintf(listing, "unknown node kind\n");

    for (i = 0; i < MAXCHILDREN; ++i)
        printTree(tree->child[i]);

    tree = tree->sibling;
}

UNINDENT;
}

```

parse.h:

```

/*****
/* File: parse.h                                     */
/* The parser interface for the C-- compiler        */
/* START: 2024.12.9                                  */
/* FINISH: 2024.12.9                                 */
*****/

#ifndef _PARSE_H_
#define _PARSE_H_

/* Function parse returns the newly
 * constructed syntax tree
 */
TreeNode* parse(void);

```

```
#endif
```

parse.c:

```
/* **** */
/* File: parse.c */
/* The parser implementation for the C-- compiler */
/* START: 2024.12.9 */
/* FINISH: 2024.12.23 */
/* **** */
```

```
#include "globals.h"
```

```
#include "util.h"
```

```
#include "scan.h"
```

```
#include "parse.h"
```

```
static TokenType token; /* holds current token */
```

```
/* function prototypes for recursive calls */
```

```
static TreeNode* declaration_list(void);
```

```
static TreeNode* declaration(void);
```

```
static TreeNode* var_declaration(void);
```

```
static TreeNode* param(void);
```

```
static TreeNode* param_list(void);
```

```
static TreeNode* compound_statement(void);
```

```
static TreeNode* local_declarations(void);
```

```
static TreeNode* statement_list(void);
```

```
static TreeNode* statement(void);
```

```
static TreeNode* expression_statement(void);
```

```
static TreeNode* if_statement(void);
```

```
static TreeNode* while_statement(void);
```

```
static TreeNode* return_statement(void);
```

```
static TreeNode* expression(void);
```

```
static TreeNode* simple_expression(TreeNode* passdown);
```

```
static TreeNode* additive_expression(TreeNode* passdown);
```

```
static TreeNode* term(TreeNode* passdown);
```

```
static TreeNode* factor(TreeNode* passdown);
```

```
static TreeNode* args(void);
```

```
static TreeNode* arg_list(void);
```

```
static TreeNode* identifier_statement(void);
```

```
static void syntaxError(char* message)
```



```

{
    fprintf(listing, "Syntax error at line %d: %s", lineno, message);
}

```

```

static void match(TokenType expected)
{
    if (token == expected)
        token = getToken();
    else
    {
        syntaxError("unexpected token ");
        printToken(token, tokenString);
        fprintf(listing, "\n");
    }
}

```

```

static ExpType matchType()
{
    ExpType t_type = Void;

    switch (token)
    {
    case INT:
        t_type = Integer;
        token = getToken();
        break;
    case VOID:
        t_type = Void;
        token = getToken();
        break;
    default:
        syntaxError("expected a type identifier but got a ");
        printToken(token, tokenString);
        fprintf(listing, "\n");
        break;
    }

    return t_type;
}

```

```

static int isAType(TokenType tok)
{
    if ((tok == INT) || (tok == VOID))
        return TRUE;
}

```

```

        else
            return FALSE;
    }

static TreeNode* declaration_list(void)
{
    TreeNode* tree;
    TreeNode* ptr;

    tree = declaration();
    ptr = tree;

    while (token != ENDOFFILE)
    {
        TreeNode* tmp;

        tmp = declaration();
        if ((tmp != NULL) && (ptr != NULL))
        {
            ptr->sibling = tmp;
            ptr = tmp;
        }
    }

    return tree;
}

static TreeNode* declaration(void)
{
    TreeNode* tree = NULL;
    ExpType declaration_type;
    char* identifier;

    declaration_type = matchType();
    identifier = copyString(tokenString);
    match(ID);

    switch (token)
    {
        case SEMI: /* variable declaration */
            tree = newDecNode(ScalarDeck);
            if (tree != NULL)
            {
                tree->variableDataType = declaration_type;
            }

```

```

        tree->name = identifier;
    }
    match(SEMI);
    break;

case LPAREN: /* function declaration */
    tree = newDecNode(FuncDecK);
    if (tree != NULL)
    {
        tree->functionReturnType = declaration_type;
        tree->name = identifier;
    }
    match(LPAREN);
    if (tree != NULL)
        tree->child[0] = param_list();
    match(RPAREN);
    if (tree != NULL)
        tree->child[1] = compound_statement();
    break;

default:
    syntaxError("unexpected token ");
    printToken(token, tokenString);
    fprintf(listing, "\n");
    token = getToken();
    break;
}

return tree;
}

static TreeNode* var_declaration(void)
{
    TreeNode* tree = NULL;
    ExpType declaration_type;
    char* identifier;

    declaration_type = matchType();
    identifier = copyString(tokenString);
    match(ID);

    if (token == SEMI)
    {
        tree = newDecNode(ScalarDecK); /* variable declaration */
    }

```

```

        if (tree != NULL)
        {
            tree->variableDataType = declaration_type;
            tree->name = identifier;
        }
        match(SEMI);
    }
    else
    {
        syntaxError("unexpected token ");
        printToken(token, tokenString);
        fprintf(listing, "\n");
        token = getToken();
    }
    return tree;
}

```

```

static TreeNode* param(void)
{
    TreeNode* tree;
    ExpType parmType;
    char* identifier;

    parmType = matchType(); /* get type of formal parameter */
    identifier = copyString(tokenString);
    match(ID);

    tree = newDecNode(ScalarDecK);

    if (tree != NULL)
    {
        tree->name = identifier;
        tree->val = 0;
        tree->variableDataType = parmType;
        tree->isParameter = TRUE;
    }

    return tree;
}

```

```

static TreeNode* param_list(void)
{
    TreeNode* tree;
    TreeNode* ptr;

```

```

TreeNode* newNode;

if (token == VOID) /* void param */
{
    match(VOID);
    return NULL;
}

tree = param();
ptr = tree;

while ((tree != NULL) && (token == COMMA)) /* mutiple params */
{
    match(COMMA);
    newNode = param();
    if (newNode != NULL)
    {
        ptr->sibling = newNode;
        ptr = newNode;
    }
}

return tree;
}

static TreeNode* compound_statement(void)
{
    TreeNode* tree = NULL;

    match(LBRACE);

    if ((token != RBRACE) && (tree = newStmtNode(CompoundK)))
    {
        if (isAType(token))
            tree->child[0] = local_declarations();
        if (token != RBRACE)
            tree->child[1] = statement_list();
    }
    match(RBRACE);

    return tree;
}

static TreeNode* local_declarations(void)

```

```

{
    TreeNode* tree = NULL;
    TreeNode* ptr;
    TreeNode* newNode;

    /* find first variable declaration, if it exists */
    if (isAType(token))
        tree = var_declaration();

    /* subsetmpuent variable declarations */
    if (tree != NULL)
    {
        ptr = tree;

        while (isAType(token))
        {
            newNode = var_declaration();
            if (newNode != NULL)
            {
                ptr->sibling = newNode;
                ptr = newNode;
            }
        }
    }

    return tree;
}

static TreeNode* statement_list(void)
{
    TreeNode* tree = NULL;
    TreeNode* ptr;
    TreeNode* newNode;

    if (token != RBRACE)
    {
        tree = statement();
        ptr = tree;

        while (token != RBRACE)
        {
            newNode = statement();
            if ((ptr != NULL) && (newNode != NULL))
            {

```

```

        ptr->sibling = newNode;
        ptr = newNode;
    }
}

return tree;
}

static TreeNode* statement(void)
{
    TreeNode* tree = NULL;

    switch (token)
    {
    case IF:
        tree = if_statement();
        break;
    case WHILE:
        tree = while_statement();
        break;
    case RETURN:
        tree = return_statement();
        break;
    case LBRACE:
        tree = compound_statement();
        break;
    case ID:
    case SEMI:
    case LPAREN:
    case NUM:
        tree = expression_statement();
        break;
    default:
        syntaxError("unexpected token ");
        printToken(token, tokenString);
        fprintf(listing, "\n");
        token = getToken();
        break;
    }

    return tree;
}

```

```
static TreeNode* expression_statement(void)
```

```
{
```

```
    TreeNode* tree = NULL;
```

```
    if (token == SEMI)
```

```
        match(SEMI);
```

```
    else if (token != RBRACE)
```

```
    {
```

```
        tree = expression();
```

```
        match(SEMI);
```

```
    }
```

```
    return tree;
```

```
}
```

```
static TreeNode* if_statement(void)
```

```
{
```

```
    TreeNode* tree;
```

```
    TreeNode* expr;
```

```
    TreeNode* ifStmt;
```

```
    TreeNode* elseStmt = NULL;
```

```
    match(IF);
```

```
    match(LPAREN);
```

```
    expr = expression();
```

```
    match(RPAREN);
```

```
    ifStmt = statement();
```

```
    if (token == ELSE)
```

```
    {
```

```
        match(ELSE);
```

```
        elseStmt = statement();
```

```
    }
```

```
    tree = newStmtNode(IfK);
```

```
    if (tree != NULL)
```

```
    {
```

```
        tree->child[0] = expr;
```

```
        tree->child[1] = ifStmt;
```

```
        tree->child[2] = elseStmt;
```

```
    }
```

```
    return tree;
```



```
}
```

```
static TreeNode* while_statement(void)
```

```
{
```

```
    TreeNode* tree;
```

```
    TreeNode* expr;
```

```
    TreeNode* stmt;
```

```
    match(WHILE);
```

```
    match(LPAREN);
```

```
    expr = expression();
```

```
    match(RPAREN);
```

```
    stmt = statement();
```

```
    tree = newStmtNode(WhileK);
```

```
    if (tree != NULL)
```

```
    {
```

```
        tree->child[0] = expr;
```

```
        tree->child[1] = stmt;
```

```
    }
```

```
    return tree;
```

```
}
```

```
static TreeNode* return_statement(void)
```

```
{
```

```
    TreeNode* tree;
```

```
    TreeNode* expr = NULL;
```

```
    match(RETURN);
```

```
    tree = newStmtNode(ReturnK);
```

```
    if (token != SEMI)
```

```
        expr = expression();
```

```
    if (tree != NULL)
```

```
        tree->child[0] = expr;
```

```
    match(SEMI);
```

```
    return tree;
```

```
}
```

```
static TreeNode* expression(void)
```

```

{
    TreeNode* tree = NULL;
    TreeNode* lvalue = NULL;
    TreeNode* rvalue = NULL;
    int gotLvalue = FALSE;

    if (token == ID)
    {
        lvalue = identifier_statement();
        gotLvalue = TRUE;
    }

    /* assign */
    if ((gotLvalue == TRUE) && (token == ASSIGN))
    {
        if ((lvalue != NULL) && (lvalue->nodekind == ExpK) &&
            (lvalue->kind.exp == IdK))
        {
            match(ASSIGN);
            rvalue = expression();
            tree = newExpNode(AssignK);
            if (tree != NULL)
            {
                tree->child[0] = lvalue; /* left  value */
                tree->child[1] = rvalue; /* right value */
            }
        }
        else
        {
            syntaxError("attempt to assign to something not an lvalue\n");
            token = getToken();
        }
    }
    else
        tree = simple_expression(lvalue);

    return tree;
}

static TreeNode* simple_expression(TreeNode* passdown)
{
    TreeNode* tree;
    TreeNode* lExpr = NULL;

```

```

TreeNode* rExpr = NULL;
TokenType operator;

lExpr = additive_expression(passdown);

if ((token == LTE) || (token == GTE) || (token == GT) ||
    (token == LT) || (token == EQ) || (token == NE))
{
    operator = token;
    match(token);
    rExpr = additive_expression(NULL);

    tree = newExpNode(OpK);
    if (tree != NULL)
    {
        tree->child[0] = lExpr;
        tree->child[1] = rExpr;
        tree->op = operator;
    }
}
else
    tree = lExpr;

return tree;
}

```

```

static TreeNode* additive_expression(TreeNode* passdown)
{
    TreeNode* tree;
    TreeNode* newNode;

    tree = term(passdown);

    while ((token == PLUS) || (token == MINUS))
    {
        newNode = newExpNode(OpK);
        if (newNode != NULL)
        {
            newNode->child[0] = tree;
            newNode->op = token;
            tree = newNode;
            match(token);
            tree->child[1] = term(NULL);
        }
    }
}

```

```

    }

    return tree;
}

static TreeNode* term(TreeNode* passdown)
{
    TreeNode* tree;
    TreeNode* newNode;

    tree = factor(passdown);

    while ((token == TIMES) || (token == OVER))
    {
        newNode = newExpNode(OpK);

        if (newNode != NULL)
        {
            newNode->child[0] = tree;
            newNode->op = token;
            tree = newNode;
            match(token);
            newNode->child[1] = factor(NULL);
        }
    }

    return tree;
}

static TreeNode* factor(TreeNode* passdown)
{
    TreeNode* tree = NULL;

    /* If the subtree in "passdown" is a Factor, pass it back. */
    if (passdown != NULL) return passdown;

    if (token == ID)
    {
        tree = identifier_statement();
    }
    else if (token == LPAREN)
    {
        match(LPAREN);
        tree = expression();
    }
}

```

```

        match(RPAREN);
    }
    else if (token == NUM)
    {
        tree = newExpNode(ConstK);
        if (tree != NULL)
        {
            tree->val = atoi(tokenString);
            tree->variableDataType = Integer;
        }
        match(NUM);
    }
    else
    {
        syntaxError("unexpected token ");
        printToken(token, tokenString);
        fprintf(listing, "\n");
        token = getToken();
    }

    return tree;
}

```

```

static TreeNode* identifier_statement(void)
{
    TreeNode* tree;
    TreeNode* expr = NULL;
    TreeNode* arguments = NULL;
    char* identifier = NULL;

    if (token == ID)
        identifier = copyString(tokenString);
    match(ID);

    if (token == LPAREN)
    {
        match(LPAREN);
        arguments = args();
        match(RPAREN);

        tree = newStmtNode(CallK);
        if (tree != NULL)
        {
            tree->child[0] = arguments;

```

```

        tree->name = identifier;
    }
}
else
{
    tree = newExpNode(IdK);
    if (tree != NULL)
    {
        tree->child[0] = expr;
        tree->name = identifier;
    }
}

return tree;
}

```

```

static TreeNode* args(void)
{
    TreeNode* tree = NULL;

    if (token != RPAREN)
        tree = arg_list();

    return tree;
}

```

```

static TreeNode* arg_list(void)
{
    TreeNode* tree;
    TreeNode* ptr;
    TreeNode* newNode;

    tree = expression();
    ptr = tree;

    while (token == COMMA)
    {
        match(COMMA);
        newNode = expression();

        if ((ptr != NULL) && (tree != NULL))
        {
            ptr->sibling = newNode;
            ptr = newNode;
        }
    }
}

```

```

        }
    }

    return tree;
}

TreeNode* parse(void)
{
    TreeNode* t;

    token = getToken();
    t = declaration_list();
    if (token != ENDOFFILE)
        syntaxError("Unexpected symbol at end of file\n");

    /* t points to the fully-constructed syntax tree */
    return t;
}

```

4.3 实验演示

求三个整数中的最大值：

C-- COMPILATION: Maximum.txt

Syntax tree:

```
Function declaration: "max()" of return type "integer"
  Scalar declaration: "x" of type "integer"
  Scalar declaration: "y" of type "integer"
  Scalar declaration: "z" of type "integer"
  Compound statement
    Scalar declaration: "biggest" of type "integer"
    Assignment
      Identifier: "biggest"
      Identifier: "x"
    IF statement
      Operator: >
        Identifier: "y"
        Identifier: "biggest"
      Assignment
        Identifier: "biggest"
        Identifier: "y"
    IF statement
      Operator: >
        Identifier: "z"
        Identifier: "biggest"
      Assignment
        Identifier: "biggest"
        Identifier: "z"
    RETURN statement
      Identifier: "biggest"
Function declaration: "main()" of return type "void"
  Compound statement
    Scalar declaration: "x" of type "integer"
    Scalar declaration: "y" of type "integer"
    Scalar declaration: "z" of type "integer"
    Scalar declaration: "biggest" of type "integer"
    Assignment
      Identifier: "x"
      Call to function "input()"
    Assignment
      Identifier: "y"
      Call to function "input()"
    Assignment
      Identifier: "z"
      Call to function "input()"
    Assignment
      Identifier: "biggest"
      Call to function "max()"
        Identifier: "x"
        Identifier: "y"
        Identifier: "z"
    Call to function "output()"
      Identifier: "biggest"
```


求从 1 到 N 的累加:

```
C-- COMPILATION: Sum.txt
```

```
Syntax tree:
```

```
Function declaration: "sum()" of return type "integer"
```

```
Scalar declaration: "n" of type "integer"
```

```
Compound statement
```

```
Scalar declaration: "result" of type "integer"
```

```
Scalar declaration: "i" of type "integer"
```

```
Assignment
```

```
Identifier: "i"
```

```
Literal constant: "1"
```

```
Assignment
```

```
Identifier: "result"
```

```
Literal constant: "0"
```

```
WHILE statement
```

```
Operator: <=
```

```
Identifier: "i"
```

```
Identifier: "n"
```

```
Compound statement
```

```
Assignment
```

```
Identifier: "result"
```

```
Operator: +
```

```
Identifier: "result"
```

```
Identifier: "i"
```

```
Assignment
```

```
Identifier: "i"
```

```
Operator: +
```

```
Identifier: "i"
```

```
Literal constant: "1"
```

```
RETURN statement
```

```
Identifier: "result"
```

```
Function declaration: "main()" of return type "void"
```

```
Compound statement
```

```
Scalar declaration: "n" of type "integer"
```

```
Scalar declaration: "s" of type "integer"
```

```
Assignment
```

```
Identifier: "n"
```

```
Call to function "input()"
```

```
Assignment
```

```
Identifier: "s"
```

```
Call to function "sum()"
```

```
Identifier: "n"
```

```
Call to function "output()"
```

```
Identifier: "s"
```

```
D:\visual studio\Compiler\x64\Debug\Compiler.exe (进程 329  
按任意键关闭此窗口. . .|
```

斐波那契代码:

```
Microsoft Visual Studio 调 × + ∨

C-- COMPILATION: Fibonacci.txt

Syntax tree:
Function declaration: "fibonacci()" of return type "integer"
  Scalar declaration: "n" of type "integer"
  Compound statement
    Scalar declaration: "sum" of type "integer"
    Scalar declaration: "i" of type "integer"
    Scalar declaration: "first" of type "integer"
    Scalar declaration: "second" of type "integer"
    Assignment
      Identifier: "sum"
      Literal constant: "0"
    Assignment
      Identifier: "i"
      Literal constant: "2"
    Assignment
      Identifier: "first"
      Literal constant: "1"
    Assignment
      Identifier: "second"
      Literal constant: "1"
    IF statement
      Operator: ==
      Identifier: "n"
      Literal constant: "1"
      Compound statement
        RETURN statement
          Identifier: "first"
    IF statement
      Operator: ==
      Identifier: "n"
      Literal constant: "2"
      Compound statement
        RETURN statement
          Identifier: "second"
      Compound statement
        WHILE statement
          Operator: <
          Identifier: "i"
          Identifier: "n"
          Compound statement
            Assignment
              Identifier: "sum"
              Operator: +
              Identifier: "first"
              Identifier: "second"
            Assignment
              Identifier: "first"
              Identifier: "second"
            Assignment
```

```

        Assignment
        Identifier: "second"
        Identifier: "sum"
        Assignment
        Identifier: "i"
        Operator: +
        Identifier: "i"
        Literal constant: "1"
    RETURN statement
    Identifier: "sum"
Function declaration: "main()" of return type "void"
Compound statement
    Scalar declaration: "n" of type "integer"
    Assignment
    Identifier: "n"
    Call to function "input()"
    Call to function "output()"
    Call to function "fibonacci()"
    Identifier: "n"
D:\visual_studio\Compiler\x64\Debug\Compiler.exe (进程 25692
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->
按任意键关闭此窗口...|

```

5.语义分析程序

5.1 主要设计和实现思路

1.创建一个能够存储和管理标识符及其声明信息的数据结构，即符号表。符号表是整个代码中所有声明过的变量和函数的记录者。在 C++ 中，变量的声明只有一种，那就是 INT，而函数的声明有两种，INT 和 VOID。其中关键数据结构有：

HashNode: 符号表中的节点，包含标识符名称、指向声明节点的指针、首次引用的行号和指向下一个节点的指针。

```

typedef struct hashNode {
    char* name;          /* the name of the identifier */
    TreeNode* declaration; /* pointer to the symbol's dec. node */
    int lineFirstReferenced; /* self-explanatory */
    struct hashNode* next; /* next node on this bucket chain */
} HashNode, * HashNodePtr;

```

Hashtable: 哈希表数组，用于快速定位标识符的存储位置。

2.实现创建符号表的主要函数是 **BuildSymbolTable**，该函数主要由四步实现：

(1)调用 **drawruler** 函数，确定符号表的格式，为后期打印符号表做好准备。该函数的主要功能，是控制符号表的输出规模，规定一个统一的格式。

(2)调用 **declarePredefines** 函数进行输入输出声明的预处理，为 C--语言的 I/O 操作，即全局变量中的 **input** 函数和 **output** 函数创建符号表结点，并将其插入符号表。

(3)调用了 **buildSymboltable2** 函数，用于正式建立符号表。该函数的输入是语法树根结点，即我们需要遍历整个语法树，来寻找何种词法单元结点，需要插入到符号表中,而只有结点类型为 **DECK**（声明结点）的结点才需要插入符号表中。在上一步语法分析中，我们已经得出了各个结点的类型，因此我们一边遍历语法树，一边判断当前遍历到的结点是否属于一个声明。而声明也分为两种，一种是变量的声明，一种是函数的声明；前者直接插入到符号表中即可；后者规则稍微多一些，如果结点检测到它自己输入声明结点，且声明类型为函数，那么就会调用之前提到过的 **drawruler** 函数，增加一张符号表，用于处理该函数内部的变量声明。对于非声明类结点，只需要略过即可，但是需要进行错误检测。

(4)最后调用了 **dumpCurrentScope** 函数，将符号表打印出来。该函数调用 **insertSymbol** 函数插入信息，包括声明的变量的名字，该变量对应的树节点和该变量所出现的行数。

3.检查语法错误，通过检查相邻语法树结点的词法属性，确保是符合常规的，具体由 **analyze.c** 中的 **typecheck** 函数实现。**type check** 直接调用了一个遍历函数 **traverse**，传入的三个参数，第一个是 **syntaxTree**，代表语法树；第二个 **nullProc** 函数，代表遍历的终止条件，即遍历到叶子结点时，语法树遍历停止；第三个函数则是 **checkNode**，它制定了语法规则分析的过程。**checkNode** 一边遍历语法树，一边检查该结点的相邻结点是否符合语法规则，如果不符合就报错。

5.2 类型检查语义分析程序代码

syntab.h:

```
/* **** */
/* File: syntab.h */
/* Symbol table interface for the C-- compiler */
/* (allows only one symbol table) */
/* START: 2024.12.23 */
/* FINISH: 2024.1.6 */
/* **** */

#ifndef SYMTAB_H
#define SYMTAB_H

#include "globals.h"

/* ****
 * Type definitions that need to be visible from the outside
 */

typedef struct hashNode {
    char* name; /* the name of the identifier */
    TreeNode* declaration; /* pointer to the symbol's dec. node */
    int lineFirstReferenced; /* self-explanatory */
    struct hashNode* next; /* next node on this bucket chain */
} HashNode, * HashNodePtr;

/* Keep track of how deep the scopes are in the symbol table - used for reporting */
extern int scopeDepth;

/* Ensures that everything is initialised and setup properly prior to first use */

void initSymbolTable();

/* Inserts line numbers and a TreeNode pointer to an identifier's declaration */

void insertSymbol(char* name, TreeNode* symbolDefNode, int lineDefined);
```

```

/* Checks to see if the given symbol is already defined in the
 * current scope
 */

```

```

int symbolAlreadyDeclared(char* name);

```

```

/* Retrieves a HashNode* pointer for a supplied identifier that
 * points to the declaration node for the identifier. If the
 * operation failed, NULL is returned.
 */

```

```

HashNodePtr lookupSymbol(char* name);

```

```

/* Dumps out the current scope in the symbol table */

```

```

void dumpCurrentScope();

```

```

/* Creates a new scope by creating a new "high water mark" on
 * the symbol table. This facilitates the destruction of all
 * symbol table entries for a local scope on a subsequent
 * call to endScope().
 */

```

```

void newScope();

```

```

/* Deletes all the local variables in the local scope by deleting
 * all symbol table entries up to a high-water mark created by
 * an earlier call to newScope().
 */

```

```

void endScope();

```

```

#endif

```

symtab.c:

```

/*****
 * File: symtab.c
 * Symbol table interface for the C-- compiler
 */

```

```

/* (allows only one symbol table) */
/* START: 2024.12.23 */
/* FINISH: 2024.1.6 */
/*****

#include "globals.h"
#include "syntab.h"
#include "util.h"

/* The maximum number of buckets in the hash table. */
#define MAXTABLESIZE 233

/* Symbol used to mark the high-water mark in the symbol table*/
#define HIGHWATERMARK "invalid"

static HashNodePtr hashtable[MAXTABLESIZE];

/* The "second list" (for lack of a better name), used to track scopes. */
static HashNodePtr secondList;

int scopeDepth;

/*****
** Prototypes for static function declarations
**
*/

/* allocateSymbolNode(): allocate a new node. If a parm is NIL it's ignored */
static HashNodePtr allocateSymbolNode(char* name,
    TreeNode* declaration,
    int lineDefined);

/* hashfunction(): takes a string and generates a hash value. */
static int hashFunction(char* key);

/* error reporting */
static void flagError(char* message);

/* used in symbol table scope dump */
static char* formatSymbolType(TreeNode* node);

/* the guts of dumpCurrentScope() */
static void dumpCurrentScope2(HashNodePtr cursor);

```

```

/*****
**   Public function definitions
**/

void initSymbolTable(void)
{
    memset(hashtable, 0, sizeof(HashNodePtr) * MAXTABLESIZE);
    secondList = NULL;
}

void insertSymbol(char* name, TreeNode* symbolDefNode, int lineDefined)
{
    char errorString[80]; /* for error reporting */

    HashNodePtr newHashNode, temp;
    int hashBucket;

    /* If the symbol already exists, flag an error */
    if (symbolAlreadyDeclared(name))
    {
        sprintf(errorString, "duplicate identifier \"%s\"", name);
        flagError(errorString);
    }
    else {

        /* Locate bucket we're using */
        hashBucket = hashFunction(name);
        DEBUG_ONLY(fprintf(listing,
            "**** insertSymbol(): bucket is %d\n", hashBucket); );

        /* Allocate and insert record on front of bucket */
        newHashNode = allocateSymbolNode(name, symbolDefNode, lineDefined);
        if (newHashNode != NULL)
        {
            temp = hashtable[hashBucket];
            hashtable[hashBucket] = newHashNode;
            newHashNode->next = temp;
        }

        /* Stick node on front of "secondList" */
        newHashNode = allocateSymbolNode(name, symbolDefNode, lineDefined);
    }
}

```



```

        if (newHashNode != NULL)
        {
            temp = secondList;
            secondList = newHashNode;
            secondList->next = temp;
        }
    }
}

```

```

/* Check to see if the symbol given by "name" is already declared in the
 *   current scope.
 */

```

```

int symbolAlreadyDeclared(char* name)
{
    int symbolFound = FALSE;
    HashNodePtr cursor;

    /* Scan "secondList" within _current_ scope for duplicate definition */
    cursor = secondList;

    while ((cursor != NULL) && (!symbolFound)
           && ((strcmp(cursor->name, HIGHWATERMARK) != 0)))
    {
        if (strcmp(name, cursor->name) == 0)
            symbolFound = TRUE;
        else
            cursor = cursor->next;
    }

    return (symbolFound);
}

```

```

HashNodePtr lookupSymbol(char* name)
{
    HashNodePtr cursor;
    int hashBucket;    /* hash bucket on which to conduct our search */
    int found = FALSE;

    hashBucket = hashFunction(name);
    cursor = hashtable[hashBucket];
}

```

```

while (cursor != NULL)
{
    if (strcmp(name, cursor->name) == 0)
    {
        found = TRUE;
        break;
    }

    cursor = cursor->next;
}

if (found == TRUE)
    return cursor;
else
    return NULL;
}

/* Use recursion to dump symbols out in "reverse-reverse" order.. i.e the
 * right way around...
 */

void dumpCurrentScope()
{
    HashNodePtr cursor;

    cursor = secondList;

    /* dump it out */
    if ((cursor != NULL) && (strcmp(HIGHWATERMARK, cursor->name)))
        dumpCurrentScope2(cursor);
}

#define IDENT_LEN 12

static void dumpCurrentScope2(HashNodePtr cursor)
{
    char paddedIdentifier[IDENT_LEN + 1];
    char* typeInformation; /* used to catch result of formatSymbolType */

    if ((cursor->next != NULL)
        && (strcmp(cursor->next->name, HIGHWATERMARK) != 0))
        dumpCurrentScope2(cursor->next);
}

```

```

/* pad identifier name */
memset(paddedIdentifier, ' ', IDENT_LEN);
memmove(paddedIdentifier, cursor->name, strlen(cursor->name));
paddedIdentifier[IDENT_LEN] = '\0';

/* output symbol table entry */
typeInformation = formatSymbolType(cursor->declaration);

fprintf(listing, "%3d   %s   %7d   %c   %s\n",
        scopeDepth,
        paddedIdentifier,
        cursor->lineFirstReferenced,
        cursor->declaration->isParameter ? 'Y' : 'N',
        typeInformation);

/* Prevent a memory leak - (bjf, 11/5/2000) */
free(typeInformation);
}

void newScope()
{
    HashNodePtr newNode, temp;

    /*
     * This function is short and sweet: we just append a "high water mark"
     * on "secondList".
     */

    newNode = allocateSymbolNode(HIGHWATERMARK, NULL, 0);
    if (newNode != NULL)
    {
        temp = secondList;
        secondList = newNode;
        secondList->next = temp;
    }
}

void endScope()
{
    /* to delete all symbols in the current scope */

    HashNodePtr hashPtr;

```

```

HashNodePtr temp; /* used in freeing HashNodes */
int hashBucket;

while ((secondList != NULL)
    && (strcmp(HIGHWATERMARK, secondList->name) != 0)
{
    /* locate this node in the hash table, delete it */
    hashBucket = hashFunction(secondList->name);
    hashPtr = hashtable[hashBucket];

    /*    INVARIANT: since symbols were inserted into the hash table _and_
    *    secondList in the same order, the name of the symbol on the
    *    front of the hash bucket _must_ be the same as the one under
    *    secondListPtr.
    */

    assert((secondList != NULL) && (hashtable[hashBucket] != NULL));
    assert(strcmp(secondList->name, hashPtr->name) == 0);

    /* delete from hash table */
    temp = hashtable[hashBucket]->next;
    free(hashtable[hashBucket]);
    hashtable[hashBucket] = temp;

    /* ... and from second list */
    temp = secondList->next;
    free(secondList);
    secondList = temp;
}

/* delete high water mark */
assert(strcmp(secondList->name, HIGHWATERMARK) == 0);
temp = secondList->next;
free(secondList);
secondList = temp;
}

/* Static function definitions */

static HashNodePtr allocateSymbolNode(char* name,
    TreeNode* declaration,
    int lineDefined)
{

```

```

HashNode* temp;

temp = (HashNode*)malloc(sizeof(HashNode));
if (temp == NULL)
{
    Error = TRUE;
    fprintf(listing,
        "... Out of memory allocating memory for symbol table\n");
}
else
{
    temp->name = copyString(name);
    temp->declaration = declaration;
    temp->lineFirstReferenced = lineDefined;
    temp->next = NULL;
}

return temp;
}

/* Power-of-two multiplier in hash function */
#define SHIFT 4

/* Code borrowed from Loudon p.522 */
static int hashFunction(char* key)
{
    int temp = 0;
    int i = 0;

    while (key[i] != '\0')
    {
        temp = ((temp << SHIFT) + key[i]) % MAXTABLESIZE;
        ++i;
    }

    return temp;
}

static void flagError(char* message)
{
    fprintf(listing, ">>> Semantic error (symbol table): %s", message);
    Error = TRUE; /* global variable to inhibit subseq. passes on error */
}

```

```

static char* formatSymbolType(TreeNode* node)
{
    char stringBuffer[100];

    if ((node == NULL) || (node->nodekind != Deck))
        strcpy(stringBuffer, "<<ERROR>>");
    else
    {
        /* node is a declaration */
        switch (node->kind.dec)
        {
            case ScalarDeck:
                sprintf(stringBuffer, "Scalar of type %s",
                    typeName(node->variableDataType));
                break;
            case FuncDeck:
                sprintf(stringBuffer, "Function with return type %s",
                    typeName(node->functionReturnType));
                break;
            default:
                strcpy(stringBuffer, "<<UNKNOWN>>");
                break;
        }
    }

    return copyString(stringBuffer);
}

```

analyze.h:

```

/*****/
/* File: analyze.h */
/* Semantic analyzer interface for C-- compiler */
/* START: 2024.12.23 */
/* FINISH: 2024.1.6 */
/*****/

#ifndef ANALYSE_H
#define ANALYSE_H

#include "globals.h"

/* Takes a syntax tree and traverses it, building a symbol table

```

```

    * and decorating all identifiers with references to their
    * declarations.
    */
void buildSymbolTable(TreeNode* syntaxTree);

/* Traverses the syntax tree and type checks the program */
void typeCheck(TreeNode* syntaxTree);

#endif

```

analyze.c:

```

/*****
/* File: analyze.c                                */
/* Semantic analyzer implementation                */
/* for the C-- compiler                           */
/* START: 2024.12.23                               */
/* FINISH: 2024.1.8                               */
*****/

#include "analyze.h"
#include "globals.h"
#include "syntab.h"
#include "util.h"

/*****
 * Module-static function declarations
 */

/* draw a ruler on the screen */
static void drawRuler(FILE* output, char* string);

/* the guts of buildSymbolTable() */
static void buildSymbolTable2(TreeNode* syntaxTree);

/* flag an error from the type checker */
static void flagSemanticError(char* str);

/* generic tree traversal routine */
static void traverse(TreeNode* syntaxTree,
    void (*preProc)(TreeNode*),

```

```

void (*postProc)(TreeNode*));

/* routine to perform the actual type check on a node */
static void checkNode(TreeNode* syntaxTree);

/* dummy do-nothing procedure used to keep traversal() happy */
static void nullProc(TreeNode* syntaxTree);

/* traverse the syntax tree, marking global variables as such */
void markGlobals(TreeNode* tree);

/* declare the C-minus "built-in" input() and output() routines */
static void declarePredefines(void);

/* type-check functions' formal parameters against actual parameters */
static int checkFormalAgainstActualParms(TreeNode* formal, TreeNode* actual);

/*****
 *   Public function definitions
 */

void buildSymbolTable(TreeNode* syntaxTree)
{
    /* Format headings */
    if (TraceAnalyze)
    {
        drawRuler(listing, "");
        fprintf(listing,
            "Scope Identifier          Line   Is a   Symbol type\n");
        fprintf(listing,
            "depth                      Decl.   parm?\n");
        fprintf(listing,
            "-----\n");
    }

    declarePredefines(); /* make input() and output() visible in globals */
    buildSymbolTable2(syntaxTree);

    /* set the isGlobal field in appropriate nodes: needed in code generator */
    markGlobals(syntaxTree);

    /* Dump the global scope, if it's asked for */
    if (TraceAnalyze)

```



```

    {
        drawRuler(listing, "GLOBALS");
        dumpCurrentScope();
        drawRuler(listing, "");
        fprintf(listing, "...Symbol table dump complete...\n");
    }
}

```

```

void typeCheck(TreeNode* syntaxTree)
{
    traverse(syntaxTree, nullProc, checkNode);
}

```

```

/*****
 * Static function declarations
 */

```

```

static void declarePredefines(void)
{
    TreeNode* input;
    TreeNode* output;
    TreeNode* temp;

    /* define "int input(void)" */
    input = newDecNode(FuncDeck);
    input->name = copyString("input");
    input->functionReturnType = Integer;
    input->expressionType = Function;

    /* define "void output(int)" */
    temp = newDecNode(ScalarDeck);
    temp->name = copyString("arg");
    temp->variableDataType = Integer;
    temp->expressionType = Integer;

    output = newDecNode(FuncDeck);
    output->name = copyString("output");
    output->functionReturnType = Void;
    output->expressionType = Function;
    output->child[0] = temp;
}

```

```

/* get input() and output() added to global scope */
insertSymbol("input", input, 0);
insertSymbol("output", output, 0);
}

static void buildSymbolTable2(TreeNode* syntaxTree)
{
    int i; /* iterate over node children */
    HashNodePtr luSymbol; /* symbol being looked up */
    char errorMessage[80];

    /* used to decorate RETURN nodes with enclosing procedure */
    static TreeNode* enclosingFunction = NULL;

    while (syntaxTree != NULL)
    {
        /*
         * Examine current symbol: if it's a declaration, insert into
         * symbol table.
         */
        if (syntaxTree->nodekind == Deck)
            insertSymbol(syntaxTree->name, syntaxTree, syntaxTree->lineno);

        /* If entering a new function, tell the symbol table */
        if ((syntaxTree->nodekind == Deck)
            && (syntaxTree->kind.dec == FuncDeck))
        {
            /* record the enclosing procedure declaration */
            enclosingFunction = syntaxTree;

            if (TraceAnalyze)
                drawRuler(listing, syntaxTree->name);

            newScope();
            ++scopeDepth;
        }

        /* If entering a compound-statement, create a new scope as well */
        if ((syntaxTree->nodekind == StmtK)
            && (syntaxTree->kind.stmt == CompoundK))
        {
            newScope();
            ++scopeDepth;
        }
    }
}

```

```

}

/* If the current node is an identifier, it needs to be checked
 * against the symbol table, and annotated with a pointer back to
 * its declaration.
 */

if (((syntaxTree->nodekind == ExpK) /* identifier reference... */
    && (syntaxTree->kind.exp == IdK))
    || ((syntaxTree->nodekind == StmtK) /* function call... */
        && (syntaxTree->kind.stmt == CallK)))
{
    DEBUG_ONLY(
        fprintf(listing,
            "*** Annotating identifier \"%s\" on line %d\n",
            syntaxTree->name, syntaxTree->lineno); );

    luSymbol = lookupSymbol(syntaxTree->name);
    if (luSymbol == NULL)
    {
        /* operation failed; say so to user */
        sprintf(errorMessage,
            "identifier \"%s\" unknown or out of scope\n",
            syntaxTree->name);
        flagSemanticError(errorMessage);
    }
    else
    {
        /* Annotate identifier tree-node with a pointer to its
         * declaration.
         */
        syntaxTree->declaration = luSymbol->declaration;
    }
}

/*
 * If the current node is a RETURN statement, we need to mark
 * it with the enclosing procedure's declaration node. This
 * information is used later by the type checker to check
 * function return types.
 */
if ((syntaxTree->nodekind == StmtK) &&
    (syntaxTree->kind.stmt == ReturnK))
{

```

```

syntaxTree->declaration = enclosingFunction;

DEBUG_ONLY(fprintf(listing,
    "*** Marking return statement on line %d with pointer to "
    "%s() declaration\n",
    syntaxTree->lineno,
    syntaxTree->declaration->name));
}

for (i = 0; i < MAXCHILDREN; ++i)
    buildSymbolTable2(syntaxTree->child[i]);

/* If leaving a scope, tell the symbol table */
if (((syntaxTree->nodekind == Deck)
    && (syntaxTree->kind.dec == FuncDeck))
    || ((syntaxTree->nodekind == StmtK)
    && (syntaxTree->kind.stmt == CompoundK)))
{
    if (TraceAnalyze)
        dumpCurrentScope();
    --scopeDepth;
    endScope();
}

/* do post-order operations here */

/* visit next sibling */
syntaxTree = syntaxTree->sibling;
}
}

```

```

static void drawRuler(FILE* output, char* string)
{
    int length;
    if (strcmp(string, "") == 0)
        length = 0;
    else
        length = strlen(string) + 2;

    fprintf(output, "Function:");
    if (length > 0) fprintf(output, " %s ", string);
    fprintf(output, "\n");
}

```

```

static void flagSemanticError(char* str)
{
    fprintf(listing, ">>> Semantic error (type checker): %s", str);
    Error = TRUE;
}

/* generic tree traversal routine */
static void traverse(TreeNode* syntaxTree,
    void (*preProc)(TreeNode*),
    void (*postProc)(TreeNode*))
{
    int i;

    while (syntaxTree != NULL)
    {
        preProc(syntaxTree);

        for (i = 0; i < MAXCHILDREN; ++i)
            traverse(syntaxTree->child[i], preProc, postProc);

        postProc(syntaxTree);

        syntaxTree = syntaxTree->sibling;
    }
}

/* Take a pair of tree nodes whose children contain definitions of formal
 * parameters, and expressions-as-actual-parameters respectively.
 *
 * Compares the lists of formal and actual parameters, returns TRUE if the
 * types and number of parameters match, FALSE otherwise.
 */

static int checkFormalAgainstActualParms(TreeNode* formal, TreeNode* actual)
{
    TreeNode* firstList;
    TreeNode* secondList;

    firstList = formal->child[0];
    secondList = actual->child[0];

```

```

while ((firstList != NULL) && (secondList != NULL))
{
    if (firstList->expressionType != secondList->expressionType)
        return FALSE;

    if (firstList) firstList = firstList->sibling;
    if (secondList) secondList = secondList->sibling;
}

if (((firstList == NULL) && (secondList != NULL))
    || ((firstList != NULL) && (secondList == NULL)))
    return FALSE;

return TRUE;
}

static void checkNode(TreeNode* syntaxTree)
{
    char errorMessage[100];

    switch (syntaxTree->nodekind)
    {
    case Deck:

        switch (syntaxTree->kind.dec)
        {
        case ScalarDeck:
            syntaxTree->expressionType = syntaxTree->variableDataType;
            break;

        case FuncDeck:
            syntaxTree->expressionType = Function;
            break;
        }

        break;

    case StmtK:

        switch (syntaxTree->kind.stmt)
        {

```

case IfK:

```
if (syntaxTree->child[0]->expressionType != Integer)
{
    sprintf(errorMessage,
        "IF-expression must be integer (line %d)\n",
        syntaxTree->lineno);
    flagSemanticError(errorMessage);
}

break;
```

case WhileK:

```
if (syntaxTree->child[0]->expressionType != Integer)
{
    sprintf(errorMessage,
        "WHILE-expression must be integer (line %d)\n",
        syntaxTree->lineno);
    flagSemanticError(errorMessage);
}

break;
```

case CallK:

```
/* Check types and numbers of formal against actual parameters */
if (!checkFormalAgainstActualParms(syntaxTree->declaration,
    syntaxTree))
{
    sprintf(errorMessage, "formal and actual parameters to "
        "function don't match (line %d)\n",
        syntaxTree->lineno);
    flagSemanticError(errorMessage);
}

/* The type of this expression (procedure invocation) is that
 * of the return type of the procedure being called
 */
syntaxTree->expressionType
    = syntaxTree->declaration->functionReturnType;;

break;
```

case ReturnK:

```
/* The expression attached to the RETURN statement (if any)
 * must have the type of the return type of the function.
 */

if (syntaxTree->declaration->functionReturnType == Integer)
{
    if ((syntaxTree->child[0] == NULL)
        || (syntaxTree->child[0]->expressionType != Integer))
    {
        sprintf(errorMessage, "RETURN-expression is either "
            "missing or not integer (line %d)\n",
            syntaxTree->lineno);
        flagSemanticError(errorMessage);
    }
}
else if (syntaxTree->declaration->functionReturnType == Void)
{
    if (syntaxTree->child[0] != NULL)
    {
        sprintf(errorMessage, "RETURN-expression must be "
            "void (line %d)\n", syntaxTree->lineno);
    }
}

break;
```

case CompoundK:

```
syntaxTree->expressionType = Void;

break;
}
```

break; /* case StmtK */

case ExpK:

```
switch (syntaxTree->kind.exp)
{
case OpK:
    /*
     * This syntactic category includes both arithmetic and relational
     * operators.
```



```

*/

/* Arithmetic operators */
if ((syntaxTree->op == PLUS) || (syntaxTree->op == MINUS) ||
    (syntaxTree->op == TIMES) || (syntaxTree->op == OVER))
{
    if ((syntaxTree->child[0]->expressionType == Integer) &&
        (syntaxTree->child[1]->expressionType == Integer))
        syntaxTree->expressionType = Integer;
    else
    {
        sprintf(errorMessage, "arithmetic operators must have "
            "integer operands (line %d)\n", syntaxTree->lineno);
        flagSemanticError(errorMessage);
    }
}

/* Relational operators */
else if ((syntaxTree->op == GT) || (syntaxTree->op == LT) ||
    (syntaxTree->op == LTE) || (syntaxTree->op == GTE) ||
    (syntaxTree->op == EQ) || (syntaxTree->op == NE))
{
    if ((syntaxTree->child[0]->expressionType == Integer) &&
        (syntaxTree->child[1]->expressionType == Integer))
        syntaxTree->expressionType = Integer;
    else
    {
        sprintf(errorMessage, "relational operators must have "
            "integer operands (line %d)\n", syntaxTree->lineno);
        flagSemanticError(errorMessage);
    }
}
else
{
    sprintf(errorMessage, "error in type checker: unknown operator"
        "(line %d)\n", syntaxTree->lineno);
    flagSemanticError(errorMessage);
}

break;

case IdK:
/*
 * Handle identifiers. We can just have arrays, scalars, or
 * array element references.

```

```

    */

    if (syntaxTree->declaration->expressionType == Integer)
    {
        if (syntaxTree->child[0] == NULL)
            syntaxTree->expressionType = Integer;
        else
        {
            /* only Arrays can be indexed */
            sprintf(errorMessage, "can't access an element in "
                "something that isn't an array (line %d)\n",
                syntaxTree->lineno);
            flagSemanticError(errorMessage);
        }
    }

    else
    {
        sprintf(errorMessage, "identifier is an illegal type "
            "(line %d)\n", syntaxTree->lineno);
        flagSemanticError(errorMessage);
    }

    break;

case ConstK:

    /* C-minus supports only integers - easy to type check. */
    syntaxTree->expressionType = Integer;

    break;

case AssignK:

    /* Variable assignment */
    if ((syntaxTree->child[0]->expressionType == Integer) &&
        (syntaxTree->child[1]->expressionType == Integer))
        syntaxTree->expressionType = Integer;
    else
    {
        sprintf(errorMessage, "both assigning and assigned expression"
            " must be integer (line %d)\n", syntaxTree->lineno);
        flagSemanticError(errorMessage);
    }
}

```

```

        break;
    }

    break; /* case ExpK */

} /* switch (syntaxTree->nodekind) */
}

void markGlobals(TreeNode* syntaxTree)
{
    TreeNode* cursor;

    DEBUG_ONLY(char scratch[80]; );

    cursor = syntaxTree;

    while (cursor != NULL)
    {
        if ((cursor->nodekind == Deck) &&
            ((cursor->kind.dec == ScalarDeck)))
        {

            DEBUG_ONLY(
                sprintf(scratch, "*** Marked %s as a global variable\n",
                    cursor->name);
                fprintf(listing, scratch);
            );

            cursor->isGlobal = TRUE;
        }

        cursor = cursor->sibling;
    }
}

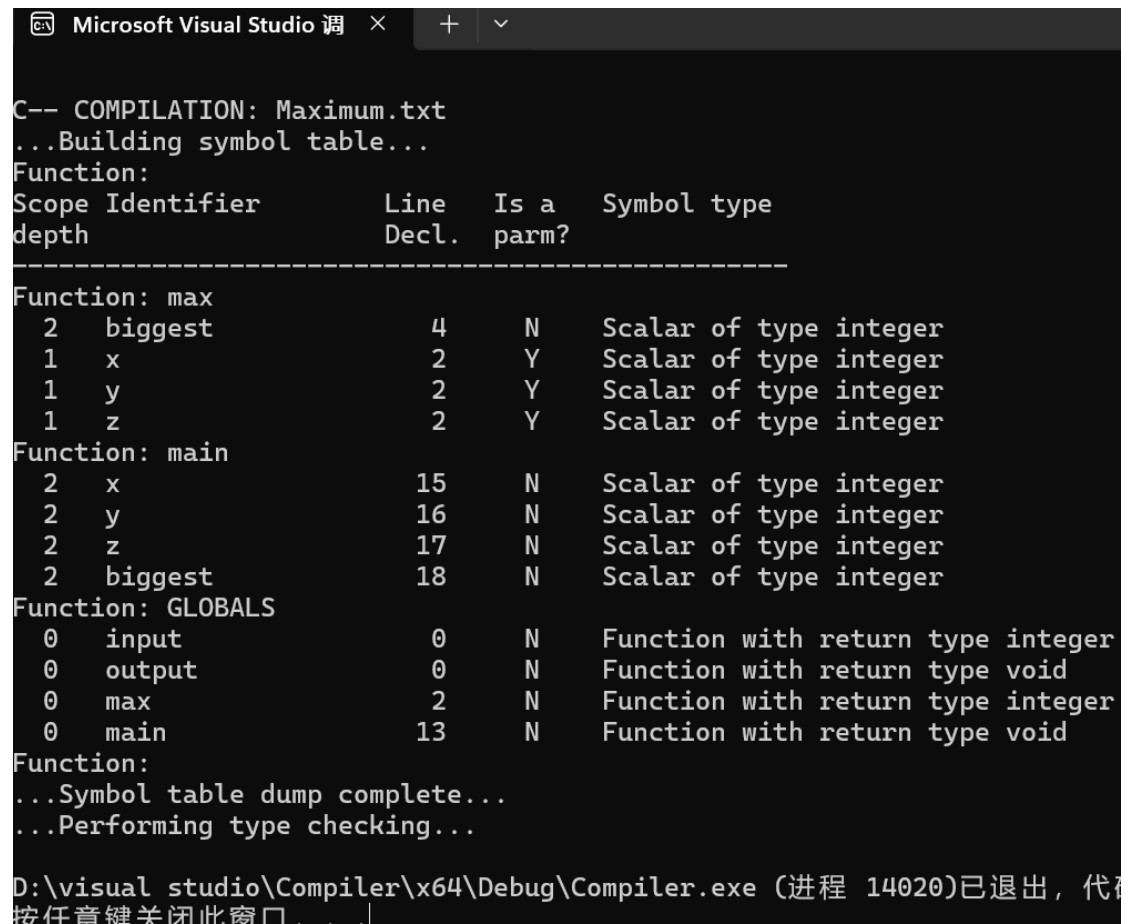
/* dummy do-nothing procedure used to keep traverse() happy */
static void nullProc(TreeNode* syntaxTree)
{
    if (syntaxTree == NULL)
        return;
    else

```

```
    return;  
}
```

5.3 实验演示

求三个整数的最大值：



```
Microsoft Visual Studio 调  ×  +  v  
C-- COMPILATION: Maximum.txt  
...Building symbol table...  
Function:  
Scope Identifier          Line  Is a  Symbol type  
depth Decl.  parm?  
-----  
Function: max  
  2  biggest              4    N   Scalar of type integer  
  1  x                    2    Y   Scalar of type integer  
  1  y                    2    Y   Scalar of type integer  
  1  z                    2    Y   Scalar of type integer  
Function: main  
  2  x                   15    N   Scalar of type integer  
  2  y                   16    N   Scalar of type integer  
  2  z                   17    N   Scalar of type integer  
  2  biggest            18    N   Scalar of type integer  
Function: GLOBALS  
  0  input                0    N   Function with return type integer  
  0  output               0    N   Function with return type void  
  0  max                  2    N   Function with return type integer  
  0  main                 13    N   Function with return type void  
Function:  
...Symbol table dump complete...  
...Performing type checking...  
D:\visual studio\Compiler\x64\Debug\Compiler.exe (进程 14020)已退出，代  
按任意键关闭此窗口
```

求从 1 到 N 的累加：

```

C-- COMPILATION: Sum.txt
...Building symbol table...
Function:
Scope Identifier      Line   Is a   Symbol type
depth                Decl.  parm?
-----
Function: sum
  2  result            3      N   Scalar of type integer
  2  i                 4      N   Scalar of type integer
  1  n                 1      Y   Scalar of type integer
Function: main
  2  n                 20     N   Scalar of type integer
  2  s                 21     N   Scalar of type integer
Function: GLOBALS
  0  input             0      N   Function with return type integer
  0  output            0      N   Function with return type void
  0  sum               1      N   Function with return type integer
  0  main              18     N   Function with return type void
Function:
...Symbol table dump complete...
...Performing type checking...

D:\visual_studio\Compiler\x64\Debug\Compiler.exe (进程 17308)已退出, 代
按任意键关闭此窗口. . .

```

斐波那契代码:

```

C-- COMPILATION: Fibonacci.txt
...Building symbol table...
Function:
Scope Identifier      Line   Is a   Symbol type
depth                Decl.  parm?
-----
Function: fibonacci
  2  sum               2      N   Scalar of type integer
  2  i                 3      N   Scalar of type integer
  2  first             4      N   Scalar of type integer
  2  second            5      N   Scalar of type integer
  1  n                 1      Y   Scalar of type integer
Function: main
  2  n                 32     N   Scalar of type integer
Function: GLOBALS
  0  input             0      N   Function with return type integer
  0  output            0      N   Function with return type void
  0  fibonacci         1      N   Function with return type integer
  0  main              31     N   Function with return type void
Function:
...Symbol table dump complete...
...Performing type checking...

D:\visual_studio\Compiler\x64\Debug\Compiler.exe (进程 33972)已退出, 代
要在调试停止时自动关闭控制台, 请启用“工具”->“选项”->“调试”->“调试停止时
按任意键关闭此窗口. . .

```

6.实验总结

本次实验巩固了我的基础知识，加深了我对编程语言语法、语义以及编译原理的理解。我成功实践了数据结构（如符号表的哈希表实现）、算法（如递归下降解析）和设计模式（如模块化编译器架构）的应用。在遇到挑战和问题时，我也通过查阅资料、求助 AI 和借鉴书中源码，提高了我分析代码解决问题的能力。编译器的构建让我从宏观上把握整个编译流程，这种系统思维对我未来的软件开发工作大有裨益。

当前任务进度小结：

(1)词法分析器：实现了将源代码转换为标记序列的功能，这是编译器能够进一步处理代码的基础。

(2)语法分析器：通过构建抽象语法树（AST），使得编译器能够理解和表示程序的结构，为后续的语义分析和代码生成提供了必要的信息。

(3)语义分析器：实现了基本的符号表构造和类型检查，确保了程序在逻辑上的一致性。

但当前的编译器仍存在很大的局限性和进步空间。比如缺失了代码生成、单独的错误处理以及优化阶段，编译器的模块化和文件交互能力也有待提升。

因此要想继续改进的话，首先应着手完成上面缺失的模块，例如优化词法分析、扩展语法规则、实现代码生成、引入优化技术、开发独立的错误处理器，并实现模块化编译以支持库的链接和使用等等。

总而言之，本实验成功实现了词法分析、语法分析和语义分析程序，尽管实现的语言类型较简单，但是作为新手的第一步也成功让我熟悉了编译器的基本原理和概念，为我深入理解编程语言和计算系统打下基础。

附： main.c

```
/* **** */
/* File: main.c */
/* Main program for C-- compiler */
/* START: 2024.11.25 */
/* FINISH: 2024.12.2 */
/* **** */
```

```
#include "globals.h"
```

```
/* set NO_PARSE to TRUE to get a scanner-only compiler */
#define NO_PARSE FALSE
/* set NO_ANALYZE to TRUE to get a parser-only compiler */
#define NO_ANALYZE FALSE
/* or to get a analyze-only compiler */
```

```
#include "util.h"
#if NO_PARSE
#include "scan.h"
#else
#include "parse.h"
#endif
#include "analyze.h"
#endif
#endif
```

```
/* allocate global variables */
int lineno = 0;
FILE* source;
FILE* listing;
FILE* code;
```

```
/* allocate and set tracing flags */
int EchoSource = FALSE;
int TraceScan = FALSE;
int TraceParse = FALSE;
int TraceAnalyze = FALSE;
```

```
int Error = FALSE;
```

```
main(int argc, char* argv[])
{
    TreeNode* syntaxTree;
```

```

char pgm[120]; /* source code file name */
if (argc != 2)
{
    fprintf(stderr, "usage: %s <filename>\n", argv[0]);
    exit(1);
}
strcpy(pgm, argv[1]);
if (strchr(pgm, '.') == NULL)
    strcat(pgm, ".tny");
source = fopen(pgm, "r");
if (source == NULL)
{
    fprintf(stderr, "File %s not found\n", pgm);
    exit(1);
}
listing = stdout; /* send listing to screen */
fprintf(listing, "\nC-- COMPILATION: %s\n", pgm);
#if NO_PARSE
    while (getToken() != ENDOFFILE);
#else
    syntaxTree = parse();
    if (TraceParse) {
        fprintf(listing, "\nSyntax tree:\n");
        printTree(syntaxTree);
    }
#endif
#if !NO_ANALYZE
    if (!Error)
    {
        fprintf(listing, "...Building symbol table...\n");
        buildSymbolTable(syntaxTree);
        fprintf(listing, "...Performing type checking...\n");
        typeCheck(syntaxTree);
    }
#endif
#endif
    fclose(source);
    return 0;
}

```